

```
!pip install timm
!pip install torch torchvision
!pip install tqdm
!pip install scikit-learn
!pip install grad-cam
```

```
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: mpmath<1.4, >=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.0) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (2.1.5)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.5.0)
Collecting grad-cam
  Downloading grad-cam-1.5.5.tar.gz (7.8 MB)
    7.8/7.8 MB 59.6 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from grad-cam) (2.0.2)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from grad-cam) (11.3.0)
Requirement already satisfied: torch>=1.7.1 in /usr/local/lib/python3.12/dist-packages (from grad-cam) (2.10.0)
Requirement already satisfied: torchvision>=0.8.2 in /usr/local/lib/python3.12/dist-packages (from grad-cam) (0.17.0)
Collecting ttach (from grad-cam)
  Downloading ttach-0.0.3-py3-none-any.whl.metadata (5.2 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from grad-cam) (4.67.3)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (from grad-cam) (4.13.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from grad-cam) (3.10.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from grad-cam) (1.6.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.7.1->grad-cam) (3.16.1)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.7.1->grad-cam) (4.12.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.7.1->grad-cam) (75.1.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.7.1->grad-cam) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.7.1->grad-cam) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.7.1->grad-cam) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.7.1->grad-cam) (2025.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->grad-cam) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->grad-cam) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->grad-cam) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->grad-cam) (1.4.7)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->grad-cam) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->grad-cam) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->grad-cam) (2.9.0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->grad-cam) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->grad-cam) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->grad-cam) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->grad-cam) (1.17.0)
Requirement already satisfied: mpmath<1.4, >=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.7.1->grad-cam) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.7.1->grad-cam) (2.1.5)
Downloading ttach-0.0.3-py3-none-any.whl (9.8 kB)
Building wheels for collected packages: grad-cam
  Building wheel for grad-cam (pyproject.toml) ... done
  Created wheel for grad-cam: filename=grad_cam-1.5.5-py3-none-any.whl size=44286 sha256=1c4108242cbb5b7e7e3d21
  Stored in directory: /root/.cache/pip/wheels/fb/3b/09/2afc520f3d69bc26ae6bd87416759c820a3f7d05c1a077bbf6
Successfully built grad-cam
Installing collected packages: ttach, grad-cam
Successfully installed grad-cam-1.5.5 ttach-0.0.3
```

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

create a validation folder

```
import os
import random
import shutil

# =====
# Dataset Paths
# =====

TRAIN_DIR = "/content/drive/MyDrive/dataset/train"
VAL_DIR = "dataset/val"
```

```
# Validation split percentage
VAL_SPLIT = 0.2

# Create validation directory
os.makedirs(VAL_DIR, exist_ok=True)

# =====
# Create validation set
# =====

# =====
# Create validation set
# =====

# Ignore hidden files like .DS_Store
classes = [
    cls for cls in os.listdir(TRAIN_DIR)
    if not cls.startswith(".")
]

for cls in classes:

    train_class_path = os.path.join(TRAIN_DIR, cls)
    val_class_path = os.path.join(VAL_DIR, cls)

    os.makedirs(val_class_path, exist_ok=True)

    images = os.listdir(train_class_path)

    # Shuffle images randomly
    random.shuffle(images)

    # Number of validation images
    val_size = int(len(images) * VAL_SPLIT)

    val_images = images[:val_size]

    train_class_path = os.path.join(TRAIN_DIR, cls)
    val_class_path = os.path.join(VAL_DIR, cls)

    os.makedirs(val_class_path, exist_ok=True)

    images = os.listdir(train_class_path)

    # Shuffle images randomly
    random.shuffle(images)

    # Number of validation images
    val_size = int(len(images) * VAL_SPLIT)

    val_images = images[:val_size]

    print(f"\nProcessing Class: {cls}")
    print(f"Moving {val_size} images to validation folder")

    for img in val_images:

        src = os.path.join(train_class_path, img)
        dst = os.path.join(val_class_path, img)

        shutil.move(src, dst)

print("\nValidation dataset created successfully.")
```

Processing Class: meningioma
Moving 212 images to validation folder

Processing Class: glioma
Moving 183 images to validation folder

```
-----
OSError                                Traceback (most recent call last)
/usr/lib/python3.12/shutil.py in move(src, dst, copy_function)
    846     try:
--> 847         os.rename(src, real_dst)
    848     except OSError:

OSError: [Errno 18] Invalid cross-device link:
'/content/drive/MyDrive/dataset/train/glioma/brisc2025_train_01141_gl_sa_t1.jpg' ->
'dataset/val/glioma/brisc2025_train_01141_gl_sa_t1.jpg'
```

During handling of the above exception, another exception occurred:

```
-----
KeyboardInterrupt                      Traceback (most recent call last)
                                         4 frames
/usr/lib/python3.12/shutil.py in _fastcopy_sendfile(fsrc, fdst)
    148     while True:
    149         try:
--> 150             sent = os.sendfile(outfd, infd, offset, blocksize)
    151         except OSError as err:
    152             # ...in order to have a more informative exception.

KeyboardInterrupt:
```

Preprocessing and store in a file

```
import os
import cv2
import numpy as np

# =====
# INPUT DATASET PATH
# =====

INPUT_DATASET = "/content/drive/MyDrive/dataset"

# =====
# OUTPUT DATASET PATH
# =====

OUTPUT_DATASET = "dataset/processed_dataset"

# =====
# IMAGE SIZE
# Higher resolution preserves MRI details
# =====

IMAGE_SIZE = 512

# =====
# CLAHE
# Improves local contrast
# =====

def apply_clahe(image):

    # Convert to grayscale
    gray = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2GRAY
    )

    # Improved CLAHE
    clahe = cv2.createCLAHE(
        clipLimit=2.0,
        tileGridSize=(8, 8)
    )

    enhanced = clahe.apply(gray)

    return enhanced

# =====
# UNSHARP MASKING
# Enhances tumor edges
```

```

# =====

def unsharp_mask(image):

    # Light Gaussian blur
    blurred = cv2.GaussianBlur(
        image,
        (3, 3),
        1.0
    )

    # Improved sharpening
    sharpened = cv2.addWeighted(
        image,
        1.3,
        blurred,
        -0.3,
        0
    )

    return sharpened

# =====
# HYBRID FILTERING
# Median + Bilateral
# Preserves edges and removes noise
# =====

def hybrid_filter(image):

    # Median filtering
    median = cv2.medianBlur(
        image,
        3
    )

    # Bilateral filtering
    bilateral = cv2.bilateralFilter(
        median,
        d=7,
        sigmaColor=50,
        sigmaSpace=50
    )

    return bilateral

# =====
# NORMALIZATION
# =====

def normalize_image(image):

    normalized = image / 255.0

    return normalized

# =====
# COMPLETE PREPROCESSING
# =====

def preprocess_image(image_path):

    # Read image
    image = cv2.imread(image_path)

    # Check image
    if image is None:

        print(f"Could not read image: {image_path}")
        return None

    # =====
    # HIGH QUALITY RESIZE
    # =====

    image = cv2.resize(
        image,
        (IMAGE_SIZE, IMAGE_SIZE),
        interpolation=cv2.INTER_CUBIC

```

```

)

# =====
# STEP 1: CLAHE
# =====

clahe_img = apply_clahe(image)

# =====
# STEP 2: UNSHARP MASK
# =====

sharp_img = unsharp_mask(clahe_img)

# =====
# STEP 3: HYBRID FILTER
# =====

filtered_img = hybrid_filter(sharp_img)

# =====
# STEP 4: NORMALIZATION
# =====

normalized_img = normalize_image(
    filtered_img
)

# Convert back for saving
final_img = (
    normalized_img * 255
).astype(np.uint8)

return final_img

# =====
# PROCESS COMPLETE DATASET
# =====

def process_dataset():

    dataset_types = [
        "train",
        "val",
        "test"
    ]

    for dtype in dataset_types:

        input_type_path = os.path.join(
            INPUT_DATASET,
            dtype
        )

        output_type_path = os.path.join(
            OUTPUT_DATASET,
            dtype
        )

        # Ignore hidden files
        classes = [
            cls for cls in os.listdir(input_type_path)
            if not cls.startswith(".")
        ]

        for cls in classes:

            input_class_path = os.path.join(
                input_type_path,
                cls
            )

            output_class_path = os.path.join(
                output_type_path,
                cls
            )

            os.makedirs(
                output_class_path,
                exist_ok=True
            )

```

```

images = [
    img for img in os.listdir(input_class_path)
    if not img.startswith(".")
]

print("\n=====")
print(f"Processing: {dtype}/{cls}")
print(f"Total Images: {len(images)}")
print("=====")

for img_name in images:

    input_image_path = os.path.join(
        input_class_path,
        img_name
    )

    output_image_path = os.path.join(
        output_class_path,
        img_name
    )

    # Preprocess image
    processed_img = preprocess_image(
        input_image_path
    )

    # Save processed image
    if processed_img is not None:

        cv2.imwrite(
            output_image_path,
            processed_img
        )

    print(f"Completed: {cls}")

print("\n=====")
print("FULL DATASET PREPROCESSING COMPLETED")
print("=====")

# =====
# MAIN
# =====

if __name__ == "__main__":

    process_dataset()

```

```

=====
Processing: train/pituitary
Total Images: 747
=====
Completed: pituitary

=====
Processing: val/meningioma
Total Images: 265
=====
Completed: meningioma

=====
Processing: val/glioma
Total Images: 229

```

```

=====
Completed: meningioma

=====
Processing: test/glioma
Total Images: 254
=====
Completed: glioma

=====
Processing: test/pituitary
Total Images: 300
=====
Completed: pituitary

=====
Processing: test/no_tumor
Total Images: 140
=====
Completed: no_tumor

=====
FULL DATASET PREPROCESSING COMPLETED
=====

```

starting the models

```

import torch
import torch.nn as nn
import timm

```

Models: CBAM

```

# =====
# CHANNEL ATTENTION
# =====

class ChannelAttention(nn.Module):

    def __init__(self, in_channels, reduction_ratio=16):

        super(ChannelAttention, self).__init__()

        self.avg_pool = nn.AdaptiveAvgPool2d(1)

        self.max_pool = nn.AdaptiveMaxPool2d(1)

        self.fc = nn.Sequential(

            nn.Conv2d(
                in_channels,
                in_channels // reduction_ratio,
                1,
                bias=False
            ),

            nn.ReLU(),

            nn.Conv2d(
                in_channels // reduction_ratio,
                in_channels,
                1,
                bias=False
            )
        )

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):

        avg_out = self.fc(
            self.avg_pool(x)
        )

        max_out = self.fc(
            self.max_pool(x)
        )

        out = avg_out + max_out

        return self.sigmoid(out)

```

```

# =====
# SPATIAL ATTENTION
# =====

class SpatialAttention(nn.Module):

    def __init__(self, kernel_size=7):

        super(SpatialAttention, self).__init__()

        self.conv = nn.Conv2d(
            2,
            1,
            kernel_size,
            padding=kernel_size // 2,
            bias=False
        )

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):

        avg_out = torch.mean(
            x,
            dim=1,
            keepdim=True
        )

        max_out, _ = torch.max(
            x,
            dim=1,
            keepdim=True
        )

        concat = torch.cat(
            [avg_out, max_out],
            dim=1
        )

        out = self.conv(concat)

        return self.sigmoid(out)

# =====
# COMPLETE CBAM
# =====

class CBAM(nn.Module):

    def __init__(self, in_channels):

        super(CBAM, self).__init__()

        self.channel_attention = ChannelAttention(
            in_channels
        )

        self.spatial_attention = SpatialAttention()

    def forward(self, x):

        x = x * self.channel_attention(x)

        x = x * self.spatial_attention(x)

        return x

```

Models: convnext_mamba

```

# =====
# SIMPLE MAMBA-STYLE BLOCK
# =====

class MambaBlock(nn.Module):

    def __init__(self, dim):

        super(MambaBlock, self).__init__()

        self.norm = nn.LayerNorm(dim)

```

```

        self.linear1 = nn.Linear(dim, dim)

        self.gelu = nn.GELU()

        self.linear2 = nn.Linear(dim, dim)

    def forward(self, x):

        residual = x

        x = self.norm(x)

        x = self.linear1(x)

        x = self.gelu(x)

        x = self.linear2(x)

        x = x + residual

        return x

```

mamba block

```

# =====
# CONVNEXT + CBAM + MAMBA
# =====

class ConvNeXtMamba(nn.Module):

    def __init__(self, num_classes=4):

        super(ConvNeXtMamba, self).__init__()

        # =====
        # CONVNEXT BACKBONE
        # =====

        self.backbone = timm.create_model(
            "convnext_tiny",
            pretrained=True,
            num_classes=0,
            in_chans=3
        )

        feature_dim = 768

        # =====
        # CBAM
        # =====

        self.cbam = CBAM(feature_dim)

        # =====
        # MAMBA BLOCK
        # =====

        self.mamba = MambaBlock(feature_dim)

        # =====
        # CLASSIFIER
        # =====

        self.classifier = nn.Sequential(

            nn.Linear(feature_dim, 256),

            nn.ReLU(),

            nn.Dropout(0.3),

            nn.Linear(256, num_classes)

        )

    def forward(self, x):

        # =====
        # Convert grayscale to pseudo RGB
        # =====

```

```

    if x.shape[1] == 1:

        x = x.repeat(1, 3, 1, 1)

    # =====
    # ConvNeXt Features
    # =====

    x = self.backbone.forward_features(x)

    # Shape:
    # [B, C, H, W]

    # =====
    # CBAM
    # =====

    x = self.cbam(x)

    # =====
    # Global Average Pooling
    # =====

    x = torch.mean(
        x,
        dim=[2, 3]
    )

    # =====
    # Mamba Block
    # =====

    x = self.mamba(x)

    # =====
    # Classification
    # =====

    x = self.classifier(x)

    return x

```

Training: Dataset loader

```

# =====
# DEVICE CONFIGURATION
# =====

import torch

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using Device:", device)

```

Using Device: cuda

```

model = ConvNeXtMamba(
    num_classes=4
)

model = model.to(device)

print(model)

```

```

    )
    )
    (norm_pre): Identity()
    (head): NormMlpClassifierHead(
      (global_pool): SelectAdaptivePool2d(pool_type=avg, flatten=Identity())
      (norm): LayerNorm2d((768,)), eps=1e-06, elementwise_affine=True)
      (flatten): Flatten(start_dim=1, end_dim=-1)
      (pre_logits): Identity()
      (drop): Dropout(p=0.0, inplace=False)
      (fc): Identity()
    )
  )
  (cbam): CBAM(
    (channel_attention): ChannelAttention(
      (avg_pool): AdaptiveAvgPool2d(output_size=1)
      (max_pool): AdaptiveMaxPool2d(output_size=1)
      (fc): Sequential(
        (0): Conv2d(768, 48, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): ReLU()
        (2): Conv2d(48, 768, kernel_size=(1, 1), stride=(1, 1), bias=False)
      )
      (sigmoid): Sigmoid()
    )
    (spatial_attention): SpatialAttention(
      (conv): Conv2d(2, 1, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), bias=False)
      (sigmoid): Sigmoid()
    )
  )
  (mamba): MambaBlock(
    (norm): LayerNorm((768,)), eps=1e-05, elementwise_affine=True)
    (linear1): Linear(in_features=768, out_features=768, bias=True)
    (gelu): GELU(approximate='none')
    (linear2): Linear(in_features=768, out_features=768, bias=True)
  )
  (classifier): Sequential(
    (0): Linear(in_features=768, out_features=256, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.3, inplace=False)
    (3): Linear(in_features=256, out_features=4, bias=True)
  )
)

```

```

# =====
# LOSS FUNCTION
# =====

criterion = nn.CrossEntropyLoss()

# =====
# OPTIMIZER
# =====

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-4,
    weight_decay=1e-4
)

# =====
# LEARNING RATE SCHEDULER
# =====

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=2
)

```

```

# =====
# ACCURACY FUNCTION
# =====

def calculate_accuracy(outputs, labels):
    _, preds = torch.max(outputs, 1)

    correct = (preds == labels).sum().item()

    accuracy = correct / labels.size(0)

    return accuracy

```

```
# =====
# TRAINING SETTINGS
# =====

EPOCHS = 20

best_val_accuracy = 0.0

train_losses = []

val_losses = []

train_accuracies = []

val_accuracies = []
```

```
!pip install tqdm
```

```
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)
```

```
from torchvision import transforms

# =====
# TRAIN TRANSFORMS
# =====

train_transforms = transforms.Compose([

    transforms.Resize((224, 224)),

    transforms.RandomHorizontalFlip(),

    transforms.RandomRotation(10),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485],
        std=[0.229]
    )
])

# =====
# VALIDATION / TEST TRANSFORMS
# =====

val_test_transforms = transforms.Compose([

    transforms.Resize((224, 224)),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485],
        std=[0.229]
    )
])
```

```
from torchvision import datasets
PROCESSED_PATH='/content/dataset/processed_dataset'

# =====
# DATASETS
# =====

train_dataset = datasets.ImageFolder(
    os.path.join(PROCESSED_PATH, "train"),
    transform=train_transforms
)

val_dataset = datasets.ImageFolder(
    os.path.join(PROCESSED_PATH, "val"),
    transform=val_test_transforms
)

test_dataset = datasets.ImageFolder(
    os.path.join(PROCESSED_PATH, "test"),
    transform=val_test_transforms
)
```

```
print("Classes:")
print(train_dataset.classes)
```

```
Classes:
['glioma', 'meningioma', 'no_tumor', 'pituitary']
```

```
from torch.utils.data import DataLoader

# =====
# DATALOADERS
# =====

train_loader = DataLoader(
    train_dataset,
    batch_size=16,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=16,
    shuffle=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=16,
    shuffle=False
)

print("Train batches:", len(train_loader))
print("Validation batches:", len(val_loader))
print("Test batches:", len(test_loader))
```

```
Train batches: 191
Validation batches: 108
Test batches: 63
```

```
from tqdm import tqdm

path = '/content/model'
# =====
# TRAINING LOOP
# =====

for epoch in range(EPOCHS):

    print(f"\nEpoch [{epoch+1}/{EPOCHS}]")

    # =====
    # TRAINING MODE
    # =====

    model.train()

    running_train_loss = 0.0

    running_train_accuracy = 0.0

    # =====
    # TRAIN LOOP
    # =====

    for images, labels in tqdm(train_loader):

        images = images.to(device)

        labels = labels.to(device)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate loss
        loss = criterion(outputs, labels)

        # Backpropagation
        loss.backward()

        # Update weights
```

```

optimizer.step()

# Metrics
running_train_loss += loss.item()

running_train_accuracy += calculate_accuracy(
    outputs,
    labels
)

# =====
# AVERAGE TRAIN METRICS
# =====

epoch_train_loss = (
    running_train_loss / len(train_loader)
)

epoch_train_accuracy = (
    running_train_accuracy / len(train_loader)
)

# =====
# VALIDATION MODE
# =====

model.eval()

running_val_loss = 0.0

running_val_accuracy = 0.0

# =====
# VALIDATION LOOP
# =====

with torch.no_grad():

    for images, labels in val_loader:

        images = images.to(device)

        labels = labels.to(device)

        outputs = model(images)

        loss = criterion(outputs, labels)

        running_val_loss += loss.item()

        running_val_accuracy += calculate_accuracy(
            outputs,
            labels
        )

# =====
# AVERAGE VALIDATION METRICS
# =====

epoch_val_loss = (
    running_val_loss / len(val_loader)
)

epoch_val_accuracy = (
    running_val_accuracy / len(val_loader)
)

# =====
# STORE METRICS
# =====

train_losses.append(epoch_train_loss)

val_losses.append(epoch_val_loss)

train_accuracies.append(epoch_train_accuracy)

val_accuracies.append(epoch_val_accuracy)

# =====
# LEARNING RATE SCHEDULER
# =====

```

```
scheduler.step(epoch_val_loss)

# =====
# PRINT METRICS
# =====

print(f"\nTrain Loss: {epoch_train_loss:.4f}")

print(f"Train Accuracy: {epoch_train_accuracy:.4f}")

print(f"Validation Loss: {epoch_val_loss:.4f}")

print(f"Validation Accuracy: {epoch_val_accuracy:.4f}")

# =====
# SAVE BEST MODEL
# =====

if epoch_val_accuracy > best_val_accuracy:

    best_val_accuracy = epoch_val_accuracy

    model_save_path = os.path.join(
        path,
        "best_model.pth"
    )

    torch.save(
        model.state_dict(),
        model_save_path
    )

    print("\nBest Model Saved")
```

Train Loss: 0.0010
 Train Accuracy: 0.9997
 Validation Loss: 0.0472
 Validation Accuracy: 0.9890

```
import matplotlib.pyplot as plt
# =====
# TRAINING LOSS GRAPH
# =====

plt.figure(figsize=(10, 5))

plt.plot(
    train_losses,
    label="Train Loss"
)

plt.plot(
    val_losses,
    label="Validation Loss"
)

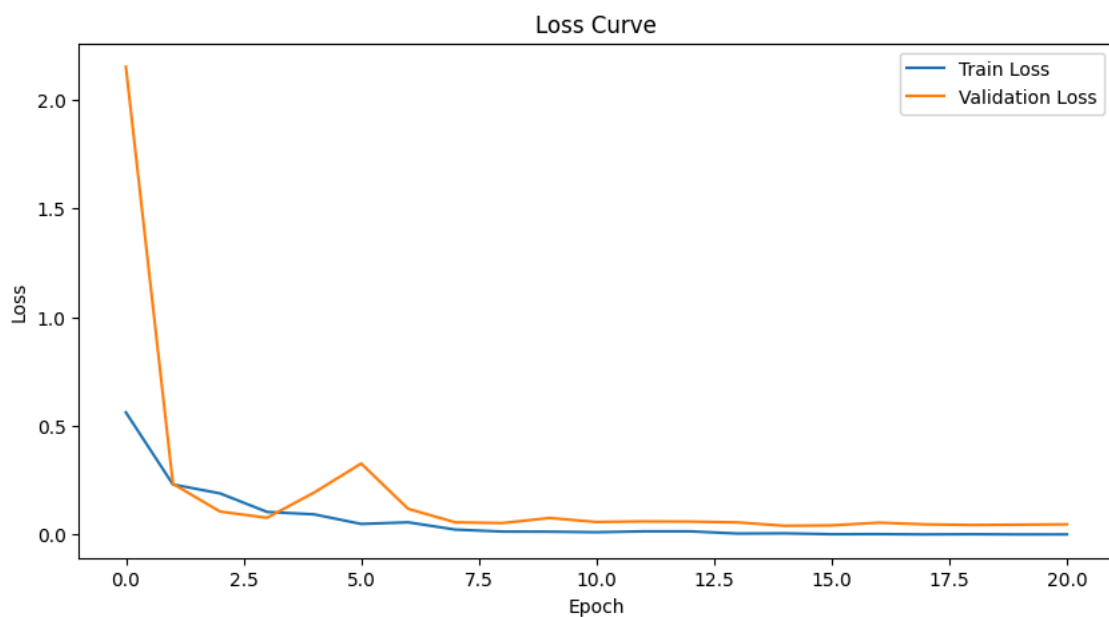
plt.title("Loss Curve")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.show()
```



```
# =====
# ACCURACY GRAPH
# =====

plt.figure(figsize=(10, 5))

plt.plot(
    train_accuracies,
    label="Train Accuracy"
)

plt.plot(
    val_accuracies,
    label="Validation Accuracy"
)

plt.title("Accuracy Curve")

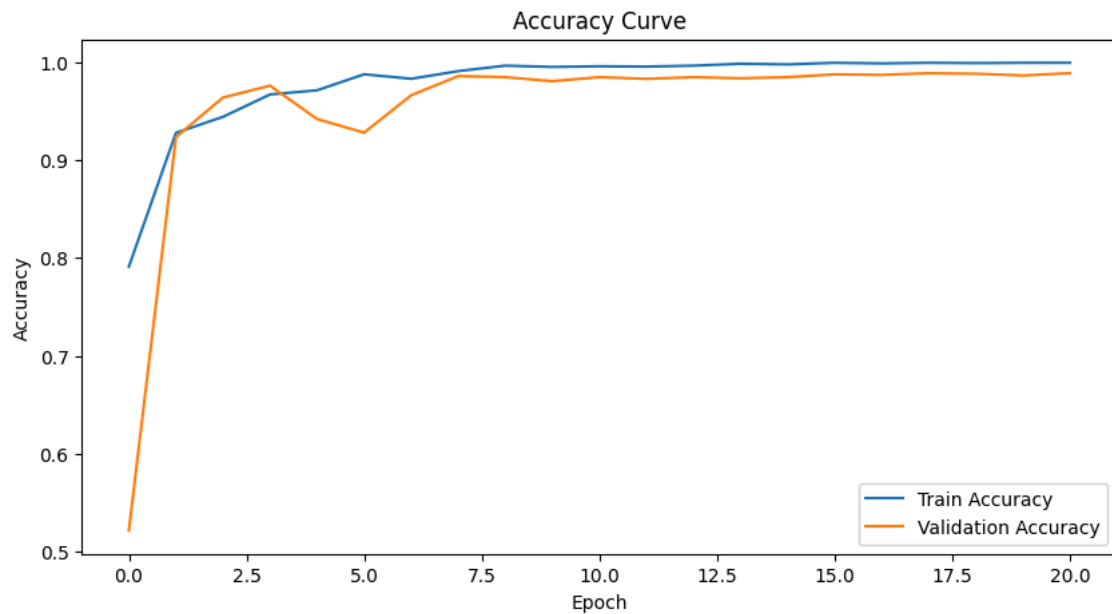
plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.show()
```

```
plt.show()
```



```
plt.figure(figsize=(10,5))

plt.plot(
    train_losses,
    label="Train Loss"
)

plt.plot(
    val_losses,
    label="Validation Loss"
)

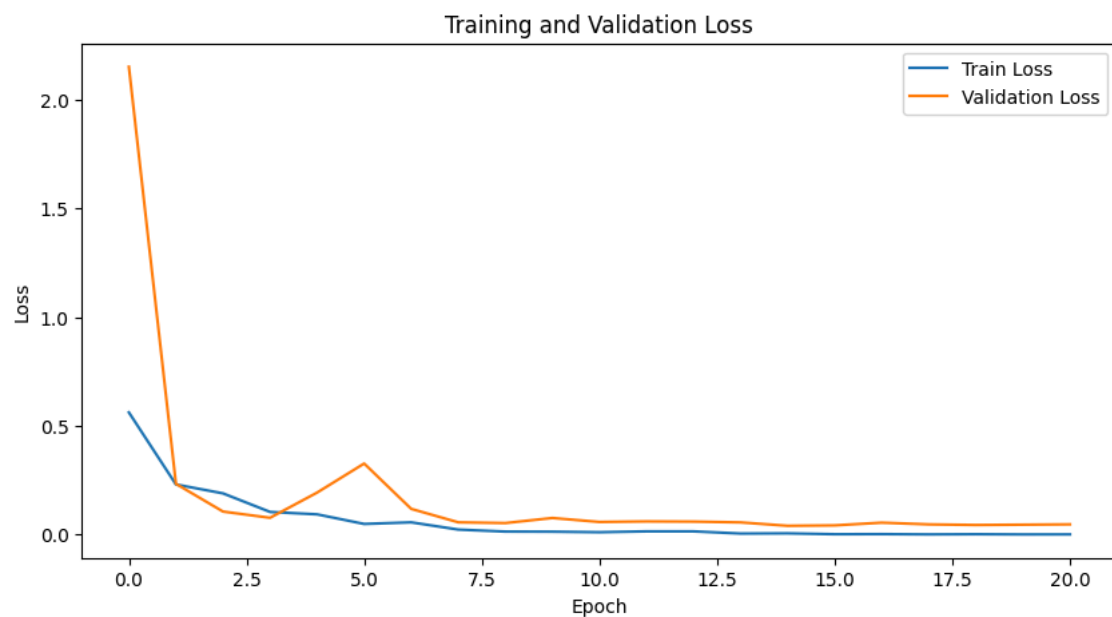
plt.title("Training and Validation Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.show()
```



```
plt.figure(figsize=(10,5))

plt.plot(
    train_accuracies,
    label="Train Accuracy"
)
```

```
plt.plot(
    val_accuracies,
    label="Validation Accuracy"
)

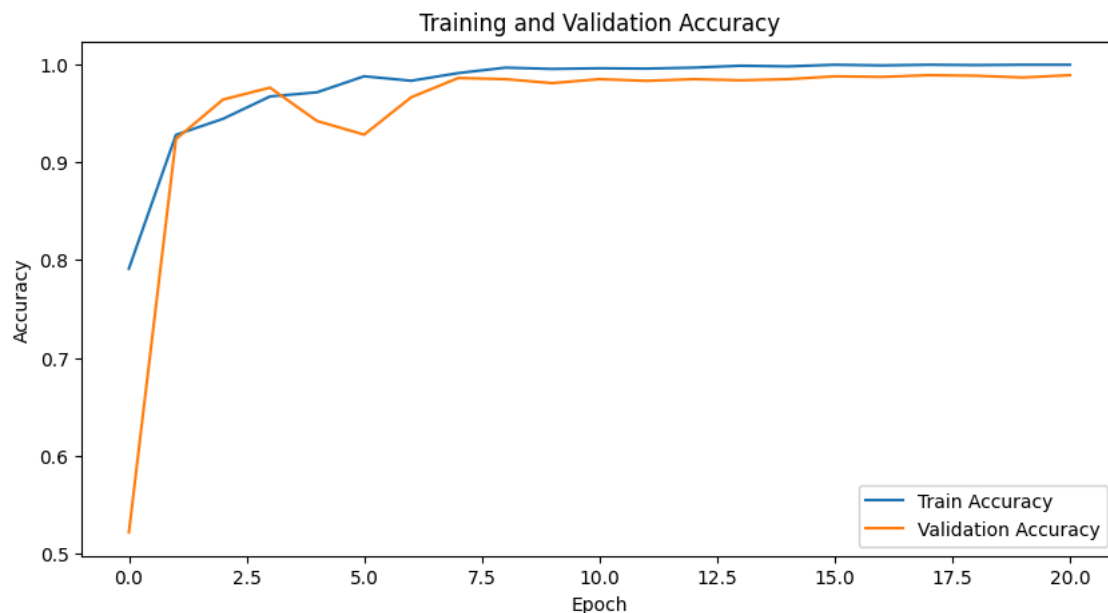
plt.title("Training and Validation Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.show()
```



```
CHECKPOINT_PATH_Model = '/content/model/'
model.load_state_dict(
    torch.load(
        os.path.join(
            CHECKPOINT_PATH_Model,
            "best_model.pth"
        )
    )
)

model.eval()

print("Best Model Loaded")
```

Best Model Loaded

```
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

all_labels = []
all_preds = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        _, preds = torch.max(outputs, 1)
        all_labels.extend(
            labels.cpu().numpy()
```

```

    )

    all_preds.extend(
        preds.cpu().numpy()
    )

# =====
# CLASSIFICATION REPORT
# =====

print("\nClassification Report:\n")

print(
    classification_report(
        all_labels,
        all_preds,
        target_names=train_dataset.classes
    )
)

# =====
# CONFUSION MATRIX
# =====

cm = confusion_matrix(
    all_labels,
    all_preds
)

plt.figure(figsize=(8,8))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=train_dataset.classes
)

disp.plot(cmap="Blues")

plt.title("Confusion Matrix")

plt.show()

```

Classification Report:

	precision	recall	f1-score	support
glioma	1.00	0.98	0.99	254
meningioma	0.98	0.99	0.99	306
no_tumor	0.99	1.00	1.00	140
pituitary	0.99	0.99	0.99	300
accuracy			0.99	1000
macro avg	0.99	0.99	0.99	1000
weighted avg	0.99	0.99	0.99	1000

<Figure size 800x800 with 0 Axes>

